

Instructor: Date:

Learner: Regional Branch:

Program: Center:

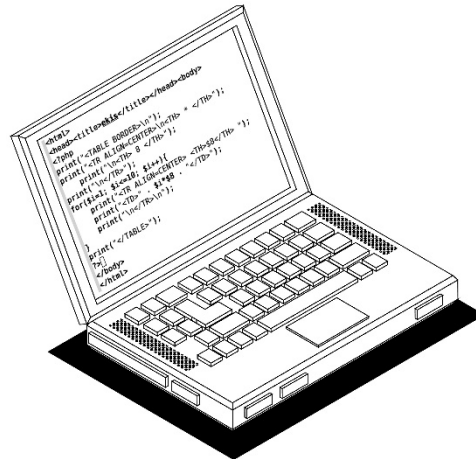
CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Introducing Scripting Languages



Learning Outcome:

Understand key information about the concepts, principles, and methodologies in simple spoken and written situations. Talk about programming techniques, use cases, kinds, and sequences in simple spoken and written situations.

Can Do Statements:

- I can talk about scripting languages
- I can obtain information about scripting languages in different contexts.
- I can exchange information about scripting languages.

Ask your partner:

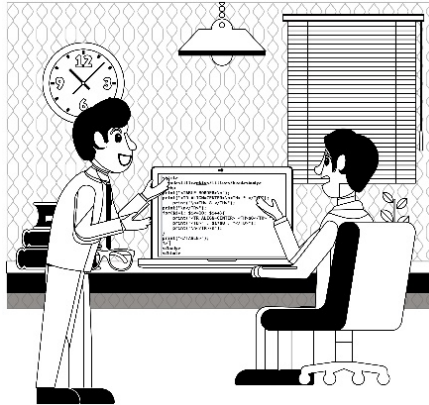
- How can we deal with repetitive tasks when executing a program?
- Do you consider necessary to have a programming code to automate common tasks? Why? Why not?





Let's Explore!

Scripting Languages offer multiple features.



Instructions: Read the text below.

Sam: Catherine, I can't understand the term "scripting language". Can you talk to me about it?

Catherine: Yes, scripting languages contain commands presented in a computer in order to carry out specific functions and provide scripts that can work on the server-side and client-side at multiple levels. As a result users can control icons, keystrokes and some other different tools as any other program features always available to perform innumerable tasks. Thus, scripting languages are in fact, programming languages.

Sam: I understand, but continue, I must know what a script is. How do I find scripting languages in context?

Catherine: A script comprises a number of text lines in a file, when a user appeals to a file, the script language processor parses each line; it means, an application programmer writes program codes in form of scripts that can be interpreted and executed one at a time. An example is: a system administrator who wants to keep record of the 'log in' times of users. He can decide to use a scripting language to facilitate the automating routine task using the multitasking and multiuser utilities it has. For such purposes, you can work with JavaScript, VBScript, AppleScript, Python, CoffeeScript and some other scripting languages.

Sam: I know that shell programming is the most common use of scripting languages.

Catherine: I think you can exchange that information with me, go ahead!

Sam: Well, shell scripting controls automating routine tasks, especially, in multiusers systems running with UNIX. Time must bring changes in programming methods and that's why programmers look for languages that combine interactive and fast nature of shells. These are structural and convenient features that can work for an effective pattern-matching, data analysis and some other tasks. This explains, the emergence of popular scripting languages such as the ones you mentioned before.



Let's Practice!

Let's talk about scripting languages.

Instructions: Proofread the sentences. Correct the mistakes. Write the corrected sentences on the lines provided.

1. Scripting languages moose interpret and execute commands in order to run an application.

2. Scripting plan work as a programming language to make programming easier, simple and effective.

3. Scripting languages tan contain server-side and/or client-side features.



Let's Practice!

Let's check different scripting languages and their main features.

Instructions: Match scripting language word with a definition. Check your answers with your partner. Discuss.

COLUMN A

1. JavaScript
2. CoffeeScript
3. VBScript
4. AppleScript
5. Python
6. Tool Command Language

COLUMN B

- a. It works on scripting levels or sides: client-side scripting and server-side scripting.
- b. This language compiles into JavaScript.
- c. It optimizes websites for Internet Explorer.
- d. Mac operative systems makes use of this.
- e. It has several features such as: interpreting, interacting and object orienting... easy to read, maintain and program.
- f. Is able to communicate in several applications.



Extension Activity

Let's look for some information about Scripting Languages.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Search for extra information about scripting languages
- Identify the main differences among scripting languages.
- Write down those differences and make a brochure to explain them.
- Share your brochure with the class.

Self-Assessment

Yes No

- I search for extra information about scripting languages.
- I Identify aspects about scripting languages.
- I write down those aspects and make a brochure with them.
- I share my brochure with the class.

☐	☐
☐	☐
☐	☐
☐	☐



Target Words

- **CoffeeScript** (noun): a type of programming language that supports JavaScript to make it brief and readable
e.g. You can work with JavaScript, VBScript, AppleScript, Python, CoffeeScript and some other scripting languages.
- **shell programming** (noun): a computer program that contains different dialects and manipulate files, execute programs and does any necessary cleanup
e.g. Shell programming is the most common use of scripting language.
- **UNIX** (noun): an operating system with multiuser and multitasking characteristics
e.g. Well, shell scripting controls automating routine tasks, especially, in multiusers systems running with UNIX.
- **keystroke** (noun): every time you press a key you perform a keystroke, typing one character on a keyboard
e.g. As a result users can control icons, keystrokes and some other different tools as any other program features.
- **appeal** (verb): to make a request of something
e.g. A script comprises a number of text lines in a file, when a user appeals to a file, the script language processor parses each line.
- **pattern-matching** (noun): examination and tracing of specific sequences of data
e.g. Convenient features that can work for an effective pattern-matching.



Learning Points

When to use Can and Must

Can and **Must** are auxiliary verbs and they are also modal verbs, we use **can** and **must** in different contexts and situations and they are very helpful verbs.

Can

We use **can** to make a reference about **abilities**, we use it to say if we are able to perform actions or not, depending on the possibilities, depending on the circumstances.

We use **can** to **give permission** or when we **ask for permission**, **can** is less formal and more common to use with friends or an acquaintance, for example.

We use **can** to make a **deduction**, when we know something in advance and then we are able to understand what possibly happens next.

Structure

Subject + Can + Verb in base form + Complement

You can test different variables.

You can't test all the variables.

Can you test scripting shells?

NOTE: In negative form we just change **Can** for **Can't**

In interrogative form we switch **Subject** and **Can**, can goes first and then next to it is the subject.

Must

We can use **must** when we want to express **prohibition or obligation**; sometimes we can **skip pending activities and do them later** if possible, this may happen with things that you **must** do, unlike with *have to*, there is no option of skipping activities, you have to do them immediately.

We can also use **must** when we want to express **something is sure**, when there is certainty that something happened or something has already happened.

We can use **must** to express **moral duty** in order to show respect.

Structure

Subject + Must + Verb in base form + Complement

*You **must** allow users to log in.*

*You **mustn't** allow user to surf on that.*

***Must** you compile data at this level?*





Learning Points

NOTE:

In negative form we just change **Must** for **Mustn't – Must Not** In interrogative form we switch **Subject** and **Must**, it goes first and then next to it is the subject.

Examples

- *I can't understand the term "scripting language". Can you talk to me about it?*
- *Users can control icons, keystrokes and some other different tools.*
- *I must know what a script is and, how do I find scripting languages in context?*
- *Programmer writes program codes in form of scripts that can be interpreted and executed one at a time.*
- *Time must bring changes in programming methods.*

Action verbs: transitive and Intransitive verbs

The **verb** is one of the most important parts of the sentence; actually there is no sentence without a verb because the verb expresses the action of the sentence, there are two types of verbs: transitive verbs and intransitive verbs.

In *transitive verbs* there is a direct object, it usually comes **after** the verb and **the direct object receives the action of the verb**, the complement in the sentence structure can work also as a direct object

Structure:

Subject + Verb (present or past tense) + Complement (Direct object)

Applications programmers create interactive features for webpages.

Create: **transitive verb**.

Interactive features: **direct object**.

In *Intransitive verbs*, in general there is nothing after the verb, it means that **there is no object or noun receiving any action**.

Sometimes it could possibly have a complement but in this case when there is no direct object we can only say that there is an action taking place, sometimes we could infer that we know what the object is in advance.

Structure:

Subject + Verb (present or past tense)

Then, the program runs.

Run: **intransitive verb**

Examples

- *Scripting languages contain commands presented in a computer.*
- *Thus, scripting languages are in fact, programming languages.*
- *When a user appeals to a file, the script language processor parses each line.*
- *I think you can exchange that information with me.*

